

Projeto: Otimização de Telemetria com Buffer Circular

Análise de Algoritmos e Sistemas Embarcados: A Anatomia da Eficiência em Sistemas Embarcados

Marlon Silva Ferreira

2026

1 Objetivo

Implementar um sistema de monitoramento para captura de amostras de sensores e transmissão via protocolo MQTT. O projeto visa contrastar empiricamente a eficiência de uma abordagem baseada em realocação dinâmica/deslocamento de memória frente a uma implementação de **Buffer Circular** de tamanho fixo.

2 Contextualização do Problema

Em sistemas embarcados como o ESP32, existe uma disparidade temporal crítica entre a **captura de dados** (μs) e a **transmissão de rede** (ms).

O uso de coleções dinâmicas ou deslocamento de arrays para gerenciar históricos de sensores gera o fenômeno do *jitter* (instabilidade temporal). Ao deslocar elementos para manter uma janela de dados, o processador consome ciclos lineares ao tamanho da amostra, fragmentando o *heap* e podendo levar ao colapso do sistema por *stack overflow* ou latência excessiva.

3 Metodologia: As Duas Vertentes

3.1 Vertente 1: A Abordagem Ineficiente (O Anti-Padrão)

- **Lógica de Dados:** Deslocamento de elementos ($O(n)$) ou uso de `realloc()` a cada nova leitura.
- **Comportamento MQTT:** Envio síncrono que bloqueia a amostragem durante a latência de rede.

3.2 Vertente 2: A Abordagem Eficiente (Buffering Circular)

- **Lógica de Dados:** Implementação de um *Ring Buffer* com índices *Head* e *Tail*.

- **Complexidade:** Operações de inserção e remoção em tempo constante, $O(1)$.
- **Comportamento MQTT:** Modelo Produtor-Consumidor; o buffer absorve a latência da rede sem interromper o sensor.

4 Análise de Escalabilidade e Estresse

A eficiência deve ser testada sob diferentes ordens de magnitude para N (amostras):

Escala (N)	Vertente 1 (Ineficiente)	Vertente 2 (Circular)
$N = 100$	Diferença imperceptível.	Performance estável.
$N = 5.000$	Atraso perceptível na amostragem.	$O(1)$ mantém latência mínima.
$N = 20.000+$	Alto consumo de CPU e <i>jitter</i> .	Sem alteração no tempo de execução.

5 Instrumentação de Código

Os alunos devem monitorar o tempo de execução e a saúde da memória:

```

1 unsigned long start = micros();
2 // Logica de insercao de dados
3 unsigned long duration = micros() - start;
4 Serial.printf("Latencia: %lu us | Heap Livre: %u bytes\n", duration, ESP
  .getFreeHeap());

```

6 Detalhamento dos Entregáveis

6.1 Entregável 1: Código-Fonte Estruturado

O código deve ser entregue em C++/Arduino, contendo ambas as implementações (Vertentes 1 e 2) para fins comparativos. É obrigatória a construção de uma estrutura ou classe para o **Buffer Circular**, demonstrando o manejo correto de índices sem o uso de funções de movimentação de memória em massa.

6.2 Entregável 2: Painel de Telemetria e Gráficos

Demonstração visual do sistema em operação. Os alunos devem apresentar:

- **Gráfico de Dados:** Visualização das amostras (em lote) via MQTT (Node-RED, Dashboard ou Serial Plotter).
- **Gráfico de Performance:** Comparativo temporal (latência em μs) entre as duas vertentes conforme o volume de dados aumenta.

6.3 Entregável 3: Relatório de Perfilamento e Análise

Documento técnico (preferencialmente em LaTeX) contendo:

- **Análise Assintótica:** Prova teórica da complexidade das operações implementadas.
- **Diagnóstico de Memória:** Registro do impacto da fragmentação de *heap* na Vertente 1 versus a estabilidade da Vertente 2.
- **Discussão:** Reflexão sobre o comportamento do sistema sob condições de instabilidade de rede (simulação de gargalo).